

Research report

Speculative Multi-Token Generation with Linguistically- Informed Constraints for Indian Languages

Completed on

 **Date**

Prepared by

Karthika N. J

Summary

Key Contributions

- Linguistically-Constrained Suffix Automaton
 - capture valid multi-token spans based on **word group boundaries**, ensuring linguistic well-formedness
 - Potentially reduce search space and prevent incoherent continuations (in draft token generation).
- Samanvaya Word Groups as Necessary Conditions
 - Multi-token prediction - to align with pre-defined word-groups

- Introduce linguistic constraint to filter or prioritise the draft outputs (possibly the target model (**Param 1** for Hindi) outputs as well)
- Speculative Decoding with Verifier-Drafter Models (may not be a stand-alone contribution - adaptation from the SAM paper)
 - **SAM drafter** - propose token continuations based on word-group constraints, and verifier - evaluates the draft tokens and decides whether to accept/reject.
 - **Aim** - to show that enforcing structural constraints in the draft stage improves fluency, coherence and speed, especially in low-resource, morphologically rich languages.
- Empirical Evaluation
 - Downstream tasks (Examples: Sentence completion, Machine translation etc)
-

Potential Sections in Paper

Next ACL ARR - [May 25th] - EMNLP (A*)

- Introduction

Motivation, challenges in Indian language generation, importance of Multi-token prediction (specifically focusing for Indian languages)
- Background/ Related Works
 - Speculative Decoding
 - Samanvaya - word grouping and the general framework
 - Multi-token prediction, Word Grouping, and Suffix Automaton
 - Generation in Indian Languages
- Methodology
 - Construction of linguistically constrained suffix automaton
 - Word-group based SAM + Eagle framework

- *{Any other linguistic priors addition, as possible}*

● Experiments

- Datasets - Samanvaya corpus for word-grouping
Other Indian language corpora for training and downstream tasks
- Baselines: airavat(7B) (AI4bharat), Param1 (2.9) Param-17B, Llama3.4
 - **Qwen-3.5(9/27/35B)**, Gemma-3(12/27B), Krutrim-2(12B), Aya
 - Qwen-2.5 (7/14/32B), Qwen-3(8/14/30)
 - Param 2.9B, Airavat 7B, Gemma 12B, Krutrim 12 B, Qwen3 14B, Qwen 3.5 - 9B
- Evaluation:
 - Speedup(tokens/sec), mean accepted tokens
 - Quality : BLEU, chrF (compared to original generation - by main model), human evaluation
 - surprisal metrics (may not work as it's not applicable for SAM-like setting),
- MT - BLEU, chrF, human evaluation
- WordGroupAware SAM-Decoding Experiments:

📁 experiments

● Result and Analysis

- Ablations: with and without word-group constraints
-

● Conclusion

● Future Work

- Further integration of linguistic priors (Structural) - dependency parse trees
- Generalisation to different languages - cross-lingual adaptability studies

Conclusions

Describe the significance of these insights to your company or client.

- Conclusion 01
- Conclusion 02
- Conclusion 03

Introduction

Explain the context for the research report. Share a top-level perspective on the subject's previous status, how it may have changed over time, and its current state. Provide the purpose of the report and the extent of research conducted.

Highlight a key point or quote

Significance

Describe why the research is relevant and important, including specific ways it can help organizations or clients moving forward.

Highlight a key point or quote

Literature review

Summary and analysis

Summarize the research findings. Highlight any key points you'll expand on later.

Highlight a key point or quote

Areas of improvement

- Identify limitations or shortcomings in existing literature and knowledge
- If applicable, explain how this report improves on existing research

Highlight a key point or quote

Methodology

Word Grouping

Rules:

[से, में, का, के, को, के लिए, पर, ने, भी, ही, द्वारा, वाला, वाली, वाले, जी, के बाद, के साथ, के बीच, के दौरान, के खिलाफ़, के प्रति, की ओर, बारे में, के मुताबिक़, के मुताबिक, के तहत, ओर से, के कारण, ने भी, मैं ही, ही मैं, सी, तरह, दी_थी]

[रहे हैं, रहा है, रही है, सकता है, सकती है, सकते हैं, हो गयी, हो गया,]

[दे दिया, दे दो, दे दें, मिला दें, मुकर जाएं, सम्मिलित करना, हाल ही में, कर दी]

With right : [numbers , नहीं]

*c + word : [ी गई, ी जाएगी, ी जायेगी, ा गया, ा जायेगा, ा जाएगा, ा चाहिए, ी चाहिए, े लगता है, े लगती है]

Unsure - [करनी है, करना है]

करना चाहिए, होना चाहिए, होनी चाहिए, होने लगता है, होने लगती है,

LLM (Llama3.1-8B-in):

Eagle

SAM - Suffix Automaton

Draft model (smaller version of LLM) +5tokens

RoPE

Description of approach

Highlight a key point or quote

Sample size and technique

- IFT DataDescribe techniques used, such as data analysis using statistical software

Highlight a key point or quote

Limitations and constraints

- Caveat potential research shortcomings
- For example, a small sample size might not be sufficient to represent all target audiences
- Another example could be limited access to proprietary data

Highlight a key point or quote

Findings

Key findings

Add bullet points for key findings, making sure to tie them to the purpose of the research. For example: “The research identified several effective strategies for mitigating wildfires, including annual brush clearance and power line maintenance.”

- Finding 01

- Finding 02

● Finding 03

Add any additional necessary details.

Highlight a key point or quote

Implications

- Explain the relevance of the key findings to this report
- Implications should be particularly significant to your client or company
- Try to limit this section to 3 or 4 concise bullets

● Implication 01

● Implication 02

● Implication 03

Insights and analysis

01

Describe a key takeaway in one sentence

- Add supporting research
- Add supporting research
- Add supporting research

02

Describe a key takeaway in one sentence

- Add supporting research
 - Add supporting research
 - Add supporting research
-

03

Describe a key takeaway in one sentence

-
- Add supporting research
 - Add supporting research
 - Add supporting research
-

“

Summarize the findings, significance, and relevance. Why are these insights important? How can these insights help clients or companies succeed?”

”

Conclusion and recommendations

Use 3 to 5 paragraphs to establish the context and summarize highlights for the report. State why the topic is important and what it means for clients and companies like yours.

Recommendations

- Share actionable steps that can be taken to deliver the benefits indicated by the research
- Each should relate to a finding and include specific examples of actions that can be taken, if appropriate
- These can range from big-picture strategy to particular tactics, as long as they are supported by the research

Step 01

Describe step 01

Step 02

Describe step 02

Step 03

Describe step 03

Experiments

Plan with a conference target (October ACL ARR or ECIR)

Objective

1. Speculative decoding - Indian language specific study (Not been done prior)
 - a. Speedup in text generation with MTP
 - b. Can include multiple Indian languages for the study [Speedup at inference]
2. Samanvaya (with SAM) → increase acceptance rate of draft tokens at the verification stage (Quality)
 - a. Languages - Hindi (Marathi)

Meeting Discussions and comments from Prof. Ganesh on the above points

1. **Beam-search for Multi-token prediction is a rudimentary implementation but inefficient. There could be other such effective/accurate but inefficient rudimentary implementations. [LookAhead, Ngram, Medusa etc]**
2. **(1) should be more effective/accurate than a more efficient implementation such as speculative decoding (using SAM, EAGLE etc)**
3. **We need to show**
 - a. **(2) is accurate enough with Samanvaya**
 - b. **and more efficient than (1) even with Samanvaya.**
4. **However, we also need to show that the accuracy gap between 2 and 1 isn't much**

Baselines

1. Airavata
Reason - Hindi-specific
2. Param
Reason - Hindi-specific
3. Llama 3.1 instruct (check Llama4 - scout)
Multi-lingual, open-source
(Token granularity - very small tokens) - need not perform our experiments

Efficient Speculative Decoding for Llama at Scale: Challenges and Solutions - <https://arxiv.org/pdf/2508.08192>

Llama 3.1 - eagle weights available

Llama 4 (scout and maverick) - eagle weights available for

Airavata and Param - training to be done

Proposal-based Experiments to be done with [Airavata], Llama 4 scout and Param.

Check the quality of Llama 4 tokens - if good enough, leave Airavata (Airavata - instruction-finetuned on OpenHaathi, based on Llama2)

Proposed Experiments

Immediate Todos

1. Llama 4 scout + Eagle (Pranav)
Check the token quality for Hindi (before 21st Aug)
2. Present Efficient Speculative Decoding for Llama at Scale: Challenges and Solutions - <https://arxiv.org/pdf/2508.08192> (Sagar)

28th August

3. Param - Eagle training needs to be done (draft model)
4. SAM + Llama4 and(or) Airavata + SAM - insights
If visualization required, meetup with Zoho team.

New doc - 6th April

Apr 6, 2026

- Dataset (training data, SAM creation data): Indic-align (wiki-chat)
- Language - Hindi
- Word group generation - rule-based
- Baselines - Updated
 1. Airavata, Krutrim 2 (12B), Aya, Qwen 2.5, Llama3.1 - Chat / Instruct
 2. Gemma 3 - Multimodal, Chat
 3. Qwen 3 - ~~Thinking~~ / Non-thinking
 4. Qwen 3.5 - Multimodal, ~~Thinking~~ / non-thinking
 5. Param1 - Instruct
 6. Param2 - Thinking - On Hold - (can try once others are complete)
- Qwen has chat variants, but you'll see the reasoning process in inference too, just like Param2. Qwen 3 and 3.5 has option to disable thinking.
Param 1-2.9B-Instruct is a chat model, and
Param2-17B-A2.4B-Thinking is a reasoning model
- Timeline
 - Airavata
 - training complete
 - Evaluation - pending (7th April)
 - Param1 - instruct
 - Training - ongoing
 - Evaluation - pending (7th April)
 - Qwen 3
-

Results

Speedup Metrics

Base Model	Drafting models/techniques	Speedup	Avg. Sequence length
Airavat	Eagle 2		
	Eagle 3		
	Eagle 2 + SAM		
	Eagle 2 + WG-Aware SAM		
Param	Eagle 2		
	Eagle 3		
	Eagle 2 + SAM		
	Eagle 2 + WG-Aware SAM		

Metrics - comparing with target model's generation (Quality analysis)

Vicuna 13b v1.3

Q → [t1 t2 t3 t4 t5 ... tk]

Temperature 0	Mean Accepted Tokens (per decoding step)	Tokens /sec	Speedup Ratio
Vanilla	1	64.63	1
Medusa	2.475	125.35	1.94
EAGLE2	4.56	165.61	2.56
EAGLE3	5.89	204.39	3.16
Temperature 0.5			

Vanilla	1	56.43	1
Medusa	2.562	121.74	2.16
EAGLE2	4.49	156.83	2.79
EAGLE3	5.7	195.69	3.47
Temperature 1			
Vanilla	1	60.95	1
Medusa	2.73	128.93	2.12
EAGLE2	4.13	148.24	2.43
EAGLE3	5.26	179.27	2.94

			Method	Draft	Accepted	Steps	Acceptance Rate	Mean Accepted Tokens (Overall)
Bias 2 Threshold 3	SAM	Static SAM	44534	1659	3181	3.73%	5.07	
		Dynamic SAM	9236	59311	6594	64.25%		
					Total Steps 18826			
	WordGroupAware SAM	Static SAM	44618	1670	3187	3.74%	4.83	
		Dynamic SAM	99974	57178	7141	57.19%		
					Total Steps 18678			
Bias 0 Threshold 0	SAM	Static SAM	233604	5956	16686	2.55%	3.24	
		Dynamic SAM	195874	65174	13991	33.27%		
					Total Steps 30677			
	WordGroupAware SAM	Static SAM	233674	5952	16691	2.55%	3.11	
		Dynamic SAM	205072	64460	14648	31.43%		

					Total Steps 31339			
--	--	--	--	--	----------------------	--	--	--

In-depth comparison between SAM-Decoding and WordGroupAware
SAM on token-level

Meeting Minutes

Internal Meeting Minutes

May 5, 2026

Apr 27, 2026

- The exact matching of generated tokens is because of the greedy matching done at the temperature 0, otherwise, the losslessness is with respect to the probability distribution.
- ToDo
- Concrete examples of greedy and non-greedy cases
 - When multiple candidate sequences are involved, how the selection is taking place
 -
- Do we need to change the acceptance criteria
 - Justification of why change required
 - Theoretically
 - Compare the quality of generations with a sample (eg. from MTBench) with different acceptance criteria
 - Temperatures: 0, (any other value)
 - Greedy
 - Rejection sampling (eagle, SAM)
 - Typical acceptance (Medusa)
 - [Objective: To understand the differences in acceptance criteria]
-

Apr 17, 2026

- With temp=0 the generation is lossless - all methods give exactly same output.
- Qn: With increased temperature, will the generations be different?
temp=1
If generations are different for different methods (need to have strong reasoning regarding why it is so), we can go ahead with previously discussed metrics of comparison.
- If not
 - Draft acceptance rate - check if there is an increase for SAM+WG compared to the rest.

-

Apr 13, 2026

- Drafts generated: w1w2w3
 - At the verification - in case of eagle:
 - Check if Eagle-based generation is same or different for Hindi in llama3.1 (as English is giving same output).
 - Confirm why the Eagle paper says the verification will be lossless (the token generation will be exactly same as the original target model - Why?)
- Update the SAM based on the latest word grouping rules.
- Metrics :
 - Similarity-based metrics (Cosine similarity, BERTScore etc.)
 - LLM-as-judge - (Gemma27B/ Qwen3-235B) - A
 - Defining scoring metrics - can use rephrasing/rewriting task scoring criteria
 - Human evaluation (Only to be done finally - smaller sample set)

-

Apr 6, 2026

Discussion Points

- Confirm Eagle 3's code issues (or in case results were worse, add those for one of the model and mention that in writing)
 - Assigned to Gokul
- Finalise the models for experiments
 - Which type? Why or why not
 - Base models
 - Chat/Instruct - Going ahead with this
 - Thinking/ Reasoning
 - ~~Multimodal~~
 - Final model list: Param 2.9B, Airavat 7B, Gemma 12B, Krutrim 12 B, Qwen3 14B, Qwen 3.5 - 9B
- If we use multimodal - will there be any issue in adding eagle 2 or SAM ?

-

- Speedup metric
- Qualitative metrics
 - generation in comparison with 'target model' -, BLEU, chrF
 - LLM as a judge - Assigned to Gokul
- Downstream task ** - slot-filling (Massive dataset), event detection. Assigned to Gokul

Immediate ToDo

1. Start write-up of related works on Spec-decoding and MTP - check literature on Indian language specific works (Karthika)
2. If we

Ideas

Draft Acceptance Criteria

Draft Acceptance Criteria

Eagle

(Added by Karthika)

Following is the algorithm used by Eagle for speculative sampling - verification process with the acceptance criteria - to determine whether the draft tokens are accepted or rejected.

Algorithm 1 Multi-round speculative sampling

Input: Target distribution p , samples t_i and distributions $\hat{p}_i$ for each i from 1 to k , where t_i is sampled from $\hat{p}_i$,

Output: a sample $x \sim p$;

$i \leftarrow 1$

for $i \leq k$ **do**

$r \leftarrow U(0, 1)$

if $r < p(t_i)/\hat{p}_i(t_i)$ **then**

Return t_i

end if

$p \leftarrow \text{norm}(\max(0, p(t_i) - \hat{p}_i(t_i)))$

$i \leftarrow i + 1$

end for

Sample $t \sim p$

Return t

This criteria constrains the generation to match exactly the same generation as the original (target) model.

This is warranted for their use-case, as the objective is to measure the speedup and acceptance rate achieved by the models, so they restrict the generations to be lossless.

[Not necessary for our use-case as our objective is to measure or verify whether the word-group aware SAM based generations have an equivalent or improved quality of the generations]

Medusa

A possible alternative, that is more relevant for our work:

However, in real-world scenarios, sampling from language models is often employed to generate diverse responses, and the temperature parameter is used merely to modulate the “creativity” of the response. Therefore, higher temperatures should result in more opportunities for the original model to accept the draft model’s output. We ascertain that it is typically unnecessary to match the distribution of the original model. Thus, we propose employing a *typical acceptance* scheme to select plausible candidates rather than using rejection sampling. This approach draws inspiration from truncation sampling studies (Hewitt et al., 2022) (refer to Appendix A for an in-depth explanation). Our objective is to choose candidates that are *typical*, meaning they are not exceedingly improbable to be produced by the original model. We use the prediction probability from the *original model* as a natural gauge for this and establish a threshold based on the prediction distribution to determine acceptance. Specifically, given $x_1, x_2, \dots, x_n$ as context, when evaluating the candidate sequence $(x_{n+1}, x_{n+2}, \dots, x_{n+K+1})$ (composed by top predictions of the original language model head and MEDUSA heads), we consider the condition

$$p_{\text{original}}(x_{n+k} | x_1, x_2, \dots, x_{n+k-1}) > \min(\epsilon, \delta \exp(-H(p_{\text{original}}(\cdot | x_1, x_2, \dots, x_{n+k-1})))) ,$$

where $H(\cdot)$ denotes the entropy function, and ϵ, δ are the hard threshold and the entropy-dependent threshold respectively. This criterion is adapted from Hewitt et al. (2022) and rests on two observations: (1) tokens with relatively high probability are meaningful, and (2) when the distribution’s entropy is high, various continuations may be deemed reasonable. During decoding, every candidate is evaluated

Draft Acceptance Criteria (Added by Gokul on 24-04-2026)

SAM is lossless under all conditions, whether $t=0$ or $t>0$. The only thing that changes is the acceptance rule. When temperature is 0, greedy acceptance is chosen. When temperature is greater than 0, the draft is accepted if it aligns with the larger model’s distribution. It needs the emitted token to be as if sampled by the larger model, meaning comparing outcomes across many runs would look the same as what the larger model alone would have produced.

In SAM decoding we have two conditions in `eval_posterior`. One is greedy decoding ($t=0$) where tokens are accepted only if they are exactly the same as the tokens generated using greedy sampling in the base model. For $t>0$, a method similar to random sampling is used for acceptance in a way that aligns with the random sampling of the baseline, so essentially

over multiple runs this also becomes lossless. For acceptance while $t > 0$, a value r is sampled from $[0,1]$ and if $p(\text{token}) \geq r$ then the token is accepted, else the token is rejected.

In the Medusa paper it states that at $t=0$ it reverts to greedy decoding, hence it becomes lossless, while at $t > 0$ a threshold is used to decide whether the draft is to be accepted or not.

In Medusa's typical acceptance the goal is simple: pick tokens that are reasonable outputs of the target model, rather than using rejection sampling. It sets a threshold based on the probability the target model assigns to the next token, which is computed during its forward pass. The draft model then proposes the next token based on its highest probability.

ϵ is a hard threshold value. δ is an entropy-dependent threshold value. The entropy of the target model's probability distribution of the next token is calculated. If the entropy is low, $\delta \cdot \exp(-H)$ becomes high, meaning the entropy term would be too strict, so ϵ caps it at a reasonable level. At high entropy, this term tends toward 0, which would be too permissive, increasing the chance of the proposed token being accepted.

In the paper they claim that even if the distribution of the draft model and the main model is the same, if the entropy is high the acceptance rate decreases. So they have incorporated the above formula where if entropy increases a lesser probable token is also accepted, else only a more probable token is accepted.

(Comments by Hrishikesh)

[22/04/26, 16:18:07] हृषीकेश राजेश तेरदाळकर: this seems a well motivated direction (relaxing lossless constraint makes sense)

[22/04/26, 16:18:48] हृषीकेश राजेश तेरदाळकर: Medusa's typical acceptance criterion could be a good candidate (would have to think a bit if there can be other candidate criteria)

[22/04/26, 16:19:00] हृषीकेश राजेश तेरदाळकर: but the entropy-adaptive threshold is particularly interesting for our setting

[22/04/26, 16:20:27] हृषीकेश राजेश तेरदाळकर: one thing..

since our SAM drafts are corpus-derived and word-group-aligned, the draft distribution is structurally different from a neural draft model like Eagle's. The typical acceptance criterion only depends on the target model's probabilities (not the draft model's)

[22/04/26, 16:20:49] हृषीकेश राजेश तेरदाळकर: this perhaps actually works in our favor, since we end up avoiding the distribution mismatch problem that makes the lossless correction expensive

[22/04/26, 16:21:41] हृषीकेश राजेश तेरदाळकर: we should do ablations on Hindi generation varying epsilon and delta

[22/04/26, 16:22:23] हृषीकेश राजेश तेरदाळकर: chrF / some kind of coherence can be evaluation metrics

[22/04/26, 16:26:15] हृषीकेश राजेश तेरदाळकर: for the ablations, epsilon and delta control quite different things but they interact, so they can't really be tuned independently

epsilon is a hard floor, no token below this probability gets accepted regardless of entropy, and delta controls how aggressively entropy relaxes the threshold

so a grid sweep makes sense, something like 3-4 values of epsilon (roughly 0.01-0.1, based on where "plausible but non-greedy" tokens sit in the target distribution) and 4-5 values of delta (0.1-0.9)

[22/04/26, 16:27:12] हृषीकेश राजेश तेरदाळकर: for each config, have to track two things simultaneously, acceptance rate (speedup proxy) and quality

the interesting metric beyond chrF could perhaps be word group boundary preservation rate among accepted tokens (should be directly computable from our existing boundary annotations, even if erroneous at least something)

[22/04/26, 16:27:52] हृषीकेश राजेश तेरदाळकर: also for baselines we need at least: Eagle strict (lossless upper bound), greedy accept-all (lower bound), and typical acceptance

if feasible, also a temperature-scaled acceptance baseline, just uniformly lowering the threshold without entropy adaptation, this would let us argue whether the entropy-adaptive part is doing real work and not just the general relaxation

[22/04/26, 16:29:10] हृषीकेश राजेश तेरदाळकर: we can run this grid on a smaller Hindi eval set first to identify the promising (epsilon, delta) region, then zoom in

Evaluation code

Medusa

<https://github.com/FasterDecoding/Medusa/blob/main/medusa/model/Utils.py>

Python

```
def evaluate_posterior(
    logits, candidates, temperature, posterior_threshold=0.3,
    posterior_alpha = 0.09, top_p=0.8, sampling = 'typical', fast = True
):
    """
    Evaluate the posterior probabilities of the candidates based on the
    provided logits and choose the best candidate.

    Depending on the temperature value, the function either uses greedy
    decoding or evaluates posterior
    probabilities to select the best candidate.

    Args:
        - logits (torch.Tensor): Predicted logits of shape (batch_size,
        sequence_length, vocab_size).
        - candidates (torch.Tensor): Candidate token sequences.
        - temperature (float): Softmax temperature for probability scaling.
        A value of 0 indicates greedy decoding.
        - posterior_threshold (float): Threshold for posterior probability.
        - posterior_alpha (float): Scaling factor for the threshold.
        - top_p (float, optional): Cumulative probability threshold for
        nucleus sampling. Defaults to 0.8.
        - sampling (str, optional): Defines the sampling strategy ('typical'
        or 'nucleus'). Defaults to 'typical'.
        - fast (bool, optional): If True, enables faster, deterministic
        decoding for typical sampling. Defaults to False.

    Returns:
        - best_candidate (torch.Tensor): Index of the chosen best candidate.
        - accept_length (int): Length of the accepted candidate sequence.
    """

    # Greedy decoding based on temperature value
    if temperature == 0:
        # Find the tokens that match the maximum logits for each
        position in the sequence
        # Argmax of logits except the last position is taken (last
        position of logits is excluded since it predicts beyond the draft
        sequence and is irrelevant), to get the token with the highest
        probability for each position.
        # A boolean comparison of this is done with each of the candidate
        draft token sequences starting from index 1.
        # This generates a mask of shape (batch_size, sequence_length -
        1) where at each position if the draft token == what the greedy
        decoding would pick, it is 1, else 0.
```

```

posterior_mask = (
    candidates[:, 1:] == torch.argmax(logits[:, :-1], dim=-1)
).int()
# Cumulative product is done along the positions of this mask.
# Once a 0 appears, everything after becomes 0 as well. Thus, if
# a token was rejected, everything after will be rejected as well.
# The sum of consecutive accepted tokens for each candidate
# sequence is calculated.
candidates_accept_length = (torch.cumprod(posterior_mask,
dim=1)).sum(dim=1)
# From the candidate sequences, the one with the longest accepted
# length is selected.
accept_length = candidates_accept_length.max()
# Choose the best candidate
if accept_length == 0:
    # Default to the first candidate if none are accepted
    best_candidate = torch.tensor(0, dtype=torch.long,
device=candidates.device)
else:
    best_candidate =
torch.argmax(candidates_accept_length).to(torch.long)
return best_candidate, accept_length

# This is the main acceptance scheme from the paper.
if sampling == 'typical':
    if fast:
        # Softmax of the logits except the last position scaled by
        # temperature is done to get the probability distribution of the vocab at
        # each position, of shape (batch_size, sequence_length - 1, vocab_size).
        posterior_prob = torch.softmax(logits[:, :-1] / temperature,
dim=-1)
        # For each position of each of the candidate sequences,
        # torch.gather is used to collect the probability of the draft token from
        # the `posterior_prob` probability distribution.
        candidates_prob = torch.gather(
            posterior_prob, dim=-1, index=candidates[:,
1:].unsqueeze(-1)
        ).squeeze(-1)
        # Entropy calculation.
        posterior_entropy = -torch.sum(
            posterior_prob * torch.log(posterior_prob + 1e-5),
dim=-1
        ) # torch.sum(torch.log(*)) is faster than torch.prod
        # If the entropy is low, `5*exp(-H)` becomes high, thus
        # this threshold would be too strict, and so `e` caps it at a reasonable
        # level.
        # At high entropy, this threshold tends toward 0 which
        # would be too permissive, which increases the chance of the
        # proposed token being accepted.
        threshold = torch.minimum(

```

```

        torch.ones_like(posterior_entropy) *
posterior_threshold,
        torch.exp(-posterior_entropy) * posterior_alpha,
    )
    # The draft token is only accepted if its probability is
higher than the threshold from above.
    posterior_mask = candidates_prob > threshold
    candidates_accept_length = (torch.cumprod(posterior_mask,
dim=1)).sum(dim=1)

    # Choose the best candidate based on the evaluated posterior
probabilities
    accept_length = candidates_accept_length.max()
    if accept_length == 0:
        # If no candidates are accepted, just choose the first
one
        best_candidate = torch.tensor(0, dtype=torch.long,
device=candidates.device)
        # If there is a tie in accept lengths of candidate
sequences, this is broken by joint log-likelihood of accepted tokens.
Argmax is done to select the highest probability prefix among these
candidates of equal length.
    else:
        best_candidates = torch.where(candidates_accept_length
== accept_length)[0]
        # Accept the best one according to likelihood
        likelihood = torch.sum(
            torch.log(candidates_prob[best_candidates,
:accept_length]), dim=-1
        )
        best_candidate =
best_candidates[torch.argmax(likelihood)]
    return best_candidate, accept_length

    # Calculate posterior probabilities and thresholds for candidate
selection
    posterior_mask = get_typical_posterior_mask(logits, candidates,
temperature, posterior_threshold, posterior_alpha, fast)
    candidates_accept_length = (torch.cumprod(posterior_mask,
dim=1)).sum(dim=1)
    # Choose the best candidate based on the evaluated posterior
probabilities
    accept_length = candidates_accept_length.max()

    if accept_length == 0:
        # If no candidates are accepted, just choose the first one
        best_candidate = torch.tensor(0, dtype=torch.long,
device=candidates.device)
    else:
        best_candidate =
torch.argmax(candidates_accept_length).to(torch.long)

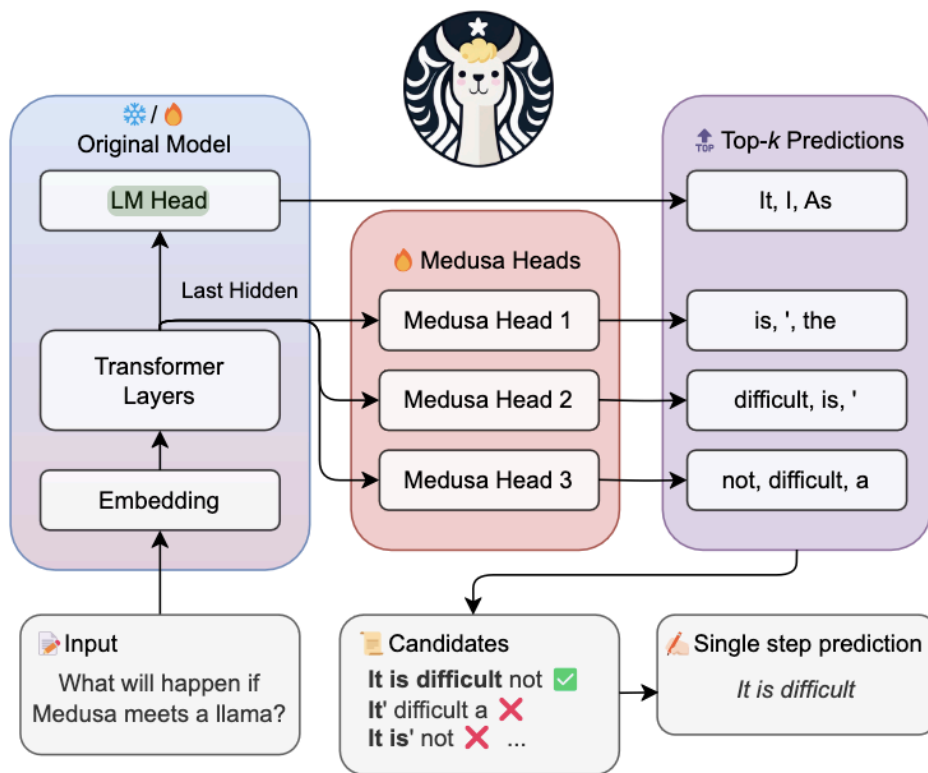
```

```

        # Accept the best one according to likelihood
        return best_candidate, accept_length

    if sampling == 'nucleus':
        assert top_p < 1.0 + 1e-6, "top_p should between 0 and 1"
        posterior_mask = get_nucleus_posterior_mask(logits, candidates,
        temperature, top_p)
        candidates_accept_length = (torch.cumprod(posterior_mask,
        dim=1)).sum(dim=1)
        accept_length = candidates_accept_length.max()
        # Choose the best candidate
        if accept_length == 0:
            # Default to the first candidate if none are accepted
            best_candidate = torch.tensor(0, dtype=torch.long,
            device=candidates.device)
        else:
            best_candidate =
            torch.argmax(candidates_accept_length).to(torch.long)
        return best_candidate, accept_length
    else:
        raise NotImplementedError

```



EAGLE

<https://github.com/SafeAILab/EAGLE/blob/main/eagle/model/utils.py>

Python

```
def evaluate_posterior(
    logits: torch.Tensor,
    candidates: torch.Tensor,
    logits_processor,
):
    """
    Evaluate the posterior probabilities of the candidates based on the
    provided logits and choose the best candidate.

    Depending on the temperature value, the function either uses greedy
    decoding or evaluates posterior
    probabilities to select the best candidate.

    Args:
        - logits (torch.Tensor): Predicted logits of shape (batch_size,
        sequence_length, vocab_size).
        - candidates (torch.Tensor): Candidate token sequences.
        - temperature (float): Softmax temperature for probability scaling.
        A value of 0 indicates greedy decoding.
        - posterior_threshold (float): Threshold for posterior probability.
        - posterior_alpha (float): Scaling factor for the threshold.

    Returns:
        - best_candidate (torch.Tensor): Index of the chosen best candidate.
        - accept_length (int): Length of the accepted candidate sequence.
    """
    # Greedy decoding based on temperature value
    if logits_processor is None:
        # Find the tokens that match the maximum logits for each
        position in the sequence
        posterior_mask = (
            candidates[:, 1:].to(logits.device) ==
            torch.argmax(logits[:, :-1], dim=-1)
        ).int()
        candidates_accept_length = (torch.cumprod(posterior_mask,
            dim=1)).sum(dim=1)
        accept_length = candidates_accept_length.max()
        # Choose the best candidate
        if accept_length == 0:
            # Default to the first candidate if none are accepted
            best_candidate = torch.tensor(0, dtype=torch.long,
            device=candidates.device)
        else:
            best_candidate =
            torch.argmax(candidates_accept_length).to(torch.long)
        return best_candidate, accept_length, logits[best_candidate,
        accept_length]

    else:
```

```

# Initialisation of some variables.
# `accept_length` starts at index 1.
# `accept_cand` is a growing tensor of all the accepted token IDs
so far, initialised with the token at index 0 of the 1st
candidate sequence.
# `best_candidate` is the index of the candidate row (tree path)
that produced the most recently accepted token.
    accept_length = 1
    accept_cand = candidates[0][:1]
    best_candidate = 0
# Iterating over the sequence length from index 1, if `i ==
accept_length`, it means that accept_length didn't increment, i.e. a
token was rejected, hence breaking the loop.
    for i in range(1, candidates.shape[1]):
        if i != accept_length:
            break
        adjustflag = False
        # An elementwise match is done for the 1st `accept_length`
number of tokens of each of the candidates with the `accept_cand`, i.e.
what has been accepted so far. With this we get a mask tensor of 1s for
indices indicating all matching candidates, and 0 for non-matching ones.
        # From this, we create a new tensor, filled with the
indices of non-zero values from `is_eq`, and the 1st one is
selected.
        is_eq = (candidates[:, :accept_length] ==
accept_cand).all(dim=1)
        fi = torch.nonzero(is_eq, as_tuple=True)[0][0]
        # From this candidate sequence, the logits predicting token
at index `i` (which will be at the index `i+1`) is taken, and then
scaled using `logits_processor`, and softmax is done to get the
probabilities from the logits.
        gt_logits = logits[fi, i - 1][None]
        gt_logits = logits_processor(None, gt_logits)[0]
        gtp = torch.softmax(gt_logits, dim=0)
        # Iterating through each candidate now, if that candidate
is in `is_eq`, i.e. has matching prefix, the draft token of each of
these filtered candidates at index `i` is taken, and is checked if it is
already present in the `candidates_set` or is a padding token. Else,
it's appended to `candidates_set`.
        candidates_set = []
        for j in range(candidates.shape[0]):
            if is_eq[j]:
                x = candidates[j, i]
                xi = x.item()
                if xi in candidates_set or xi == -1:
                    continue
                candidates_set.append(xi)
        # This is the rejection sampling part.
        # A random number in range [0.0, 1.0] is generated.

```

```

        # The above selected token's probability from the
distribution of the main model is taken and if it's greater than the
random value, it's accepted. (Since qx=1.0, px/qx = px).
        # Accept length is incremented by 1, and the current
candidate becomes the `best_candidate` for now.
        # The inner loop is broken as well if the token is
accepted.

        r = random.random()
        px = gtp[xi]
        qx = 1.0
        acp = px / qx
        if r <= acp:
            accept_cand = torch.cat((accept_cand, x[None]),
dim=0)

            accept_length += 1
            best_candidate = j
            break

        # If rejected, then the probability of this token in
the original probability distribution is set as 0, and then the entire
distribution is renormalised for the sum to equate to 1.0.
        # `adjustflag` indicates this renormalisation.
        else:
            gtp[xi] = 0
            gtp = gtp / gtp.sum()
            adjustflag = True

        # If there has been adjustments to the original probability
distribution, and also if the current `accept_length` is not equal to
the candidates' length, i.e. full sequence was not accepted, the
normalised distribution is selected as `sample_p`, so that the bonus
tokens are sampled from this corrected distribution.
        # If there was no rejection, we take fresh logits of the main
model at the last accepted position, and use this for `sample_p`.
        if adjustflag and accept_length != candidates.shape[1]:
            sample_p = gtp
        else:
            gt_logits = logits[best_candidate, accept_length - 1][None]
            gt_logits = logits_processor(None, gt_logits)[0]
            sample_p = torch.softmax(gt_logits, dim=0)
        return torch.tensor(best_candidate), accept_length - 1, sample_p

```

SAM-Decoding

<https://github.com/hyx1999/SAM-Decoding/blob/main/samd/utils.py>

Python

```

def eval_posterior(
    logits: torch.Tensor,

```

```

candidates: torch.Tensor,
config: SamdGenerationConfig,
):
    """
    Evaluate the posterior probabilities of the candidates based on the
    provided logits and choose the best candidate.

    Depending on the temperature value, the function either uses greedy
    decoding or evaluates posterior
    probabilities to select the best candidate.

    Args:
    - logits (torch.Tensor): Predicted logits of shape (batch_size,
    sequence_length, vocab_size).
    - candidates (torch.Tensor): Candidate token sequences.

    Returns:
    - best_candidate (torch.Tensor): Index of the chosen best candidate.
    - accept_length (int): Length of the accepted candidate sequence.
    """
    if config.greedy:
        # Greedy decoding based on temperature value
        # Find the tokens that match the maximum logits for each
        position in the sequence
        posterior_mask = (
            candidates[:, 1:] == torch.argmax(logits[:, :-1], dim=-1)
        ).int()
        candidates_accept_length = (torch.cumprod(posterior_mask,
dim=1)).sum(dim=1)
        accept_length = candidates_accept_length.max()
        # Choose the best candidate
        if accept_length == 0:
            # Default to the first candidate if none are accepted
            best_candidate = torch.tensor(0, dtype=torch.long,
device=candidates.device)
        else:
            best_candidate =
torch.argmax(candidates_accept_length).to(torch.long)
        return best_candidate, accept_length + 1, logits[best_candidate,
accept_length].view(1, -1)
    else:
        accept_length = 1
        accept_cand = candidates[0][:1]
        best_candidate = 0
        for i in range(1, candidates.shape[1]):
            if i != accept_length:
                break
            adjustflag = False
            is_eq = (candidates[:, :accept_length] ==
accept_cand).all(dim=1)

```

```

fi = torch.nonzero(is_eq, as_tuple=True)[0][0]
gt_logits = logits[fi, i - 1][None]
gt_logits = config.logits_processor(None, gt_logits)[0]
gtp = torch.softmax(gt_logits, dim=0)
candidates_set = []
for j in range(candidates.shape[0]):
    if is_eq[j]:
        x = candidates[j, i]
        xi = x.item()
        if xi in candidates_set or xi == -1:
            continue
        candidates_set.append(xi)
        r = random.random()
        px = gtp[xi]
        qx = 1.0
        acp = px / qx
        if r <= acp:
            accept_cand = torch.cat((accept_cand, x[None]),
dim=0)

            accept_length += 1
            best_candidate = j
            break
        else:
            gtp[xi] = 0
            gtp = gtp / gtp.sum()
            adjustflag = True
    if adjustflag and accept_length != candidates.shape[1]:
        sample_p = gtp
        # The sampling process is similar to that of EAGLE-2.
        # One difference is softmaxing on raw logits without temperature
        application beforehand.
    else:
        gt_logits = logits[best_candidate, accept_length - 1]
        sample_p = torch.softmax(gt_logits, dim=0)
        sample_p = sample_p.view(1, -1)
        accept_length = torch.tensor(accept_length, dtype=torch.long,
device=candidates.device)
        best_candidate = torch.tensor(best_candidate, dtype=torch.long,
device=candidates.device)
    return best_candidate, accept_length, sample_p

```

(Comments added by Harikrishnan)

Tab 9

Case 1: Greedy mode (`logits_processor` is `None`)

candidates =

```
[
  [101, 5, 7, 9],
  [101, 5, 8, 2],
  [101, 6, 3, 4]
]
```

Assume `argmax(logits)` gives:

predicted tokens =

```
[
  5, 7, 9 # for positions 1,2,3
]
```

Step 1: Compare candidates with model predictions

Check `candidates[:, 1:] == argmax(logits[:, :-1])`

posterior_mask =

```
[
  [1, 1, 1], # full match
  [1, 0, 0], # mismatch at position 2
  [0, 0, 0] # mismatch immediately
]
```

Step 2: Prefix acceptance (via `cumprod`)

`cumprod` →

```
[
  [1, 1, 1] → sum = 3
  [1, 0, 0] → sum = 1
  [0, 0, 0] → sum = 0
]
```

`candidates_accept_length` = [3, 1, 0]

`accept_length` = `max` = 3

`best_candidate` = `index 0`

`accept_length` = 0 → default to candidate 0

Case 2: Sampling mode (`logits_processor != None`)

candidates =

```
[  
  [101, 5, 7],  
  [101, 5, 8],  
  [101, 6, 3]  
]
```

Initialisation

accept_cand = [101]

accept_length = 1

Iteration i = 1

We look at token position 1.

Find candidates matching accepted prefix

prefix = [101]

matching candidates:

→ all 3 candidates match

Pick first matching candidate (fi)

fi = 0 → use logits from candidate 0

gtp = softmax(logits[fi])

Assume $P(5)=0.6$, $P(6)=0.4$

Try candidates one by one

Candidate 0 → token = 5

px = 0.6

qx = 1.0

acp = 0.6

Draw random:

- if $r \leq 0.6$ → accept
- else → reject

Suppose accepted:

```
accept_cand = [101, 5]
accept_length = 2
best_candidate = 0
-----
```

Iteration i = 2

Now prefix = [101, 5]

Matching candidates:

```
[101,5,7]
[101,5,8]
```

Let new probabilities: $P(7)=0.3$, $P(8)=0.7$

Try candidates

Candidate 0 → token 7

acp = 0.3

If rejected:

- set $P(7)=0$
- renormalize → now $P(8)=1.0$

Candidate 1 → token 8

acp = 1.0 → always accept

```
accept_cand = [101, 5, 8]
best_candidate = 1
accept_length = 3
```

```
gtp[xi] = 0
gtp = gtp / gtp.sum()
```

Word group aware

SAM

Word group aware SAM

Proof of Concept:

Using SpecBench:

Speculative Decoding + word grouping

Is the generation quality with word-group aware SAM better than regular SAM.

Objective

1. Acceptance rate could be higher.
2. Generation equivalent to target model - (whether word-group aware model is better than the other)
3. Generation better than target model ? (Cannot work with acceptance criteria designed to match target model's distribution.)
4. Use-cases: Tasks involving multi-word predictions: Named Entity Recognition, Dependency, Massive dataset (slot-filling dataset), Event extraction.

StaticSAM Build

Static SAM Build Logic

Static SAM in your code is a classic Suffix Automaton (SAM) built incrementally, one token at a time.

https://github.com/hyx1999/SAM-Decoding/blob/main/samd/sam/static_sam.py

1. Initialize automaton state
2. For each token, extend automaton by one character
3. Maintain suffix links and transitions
4. Clone states only when a structural split is required

Class: **StaticSAM**

Represents a Static Suffix Automaton used for storing token sequences and generating draft predictions efficiently.

Nested Dataclass: **SAMState**

Represents a single state/node in the suffix automaton.

Attribute	Type	Significance
<code>next</code>	<code>Dict[int, int]</code>	Stores transitions from the current state using tokens as edges to next states.
<code>link</code>	<code>int</code>	Suffix link pointing to the longest proper suffix state.
<code>length</code>	<code>int</code>	Length of the longest substring represented by the state.
<code>min_endpos</code>	<code>int</code>	Minimum ending position of substrings represented by the state in the input sequence.

Simple intuition:

Think of each **SAMState** as a "position marker" in the automaton.

next: A dictionary mapping token → next_state_index. Example: next[5] = 3 means "if you see token 5, go to state 3."

link: A "fallback pointer" to another state. When matching fails, you jump here to try shorter suffixes. Like a shortcut to the previous smaller context.

length: The longest string that ends at this state. If length=5, the longest complete pattern here has 5 tokens.

min_endpos: The position (index) in input_ids where the shortest string at this state ends. Used to extract predictions.

Attributes of StaticSAM

Attribute	Type	Significance
n_predicts	int	Maximum number of draft tokens to predict during generation.
states	List[SAMState]	Stores all states/nodes of the suffix automaton.
input_ids	List[int]	Maintains the full token history used to build the automaton.
last	int	Index of the most recently added state.
max_length	int	Length of the complete sequence inserted into the SAM.
cur_index	int	Current active state during traversal/querying.
cur_length	int	Current matched substring length during traversal/querying.

Important Methods and Their Purpose

Method	Significance
reset()	Resets traversal/query state for a new query sequence.
add_batch_tokens()	Builds SAM from multiple token sequences.
build()	Static helper method to construct SAM from batch token inputs.
expand_state()	Appends a new state to the automaton and returns its index.
add_state()	Inserts a new token into the SAM while maintaining suffix links and transitions.

<code>transfer_state()</code>	Traverses the SAM using a token transition from a given state.
<code>transfer_cur_state()</code>	Updates the current traversal state using a token.
<code>add_tokens()</code>	Adds multiple tokens sequentially into the SAM.
<code>transfer_tokens()</code>	Traverses/query-matches a sequence without modifying the SAM.
<code>lookup()</code>	Tests transition for a token from current state without updating internal state.
<code>to_anc()</code>	Moves to an ancestor suffix state satisfying prediction constraints.
<code>gen_draft()</code>	Generates draft token predictions using stored token history.

`add_state(token)`: Line by Line Explanation

Python

```
def add_state(self, token: int):
    self.max_length += 1
    # Increment global counter. Next new state will have length = 1,2,3... etc

    cur = self.expand_state(
        StaticSAM.SAMState(
            next={}, link=-1,
            length=self.max_length,      # This state represents max_length characters
            min_endpos=self.max_length,  # Shortest string ends at position max_length
        )
    )
    # CREATE new state 'cur' (will have index = len(states) - 1 after appending)
    # cur is initially empty (no outgoing transitions)
    # cur.link = -1 (not yet connected to suffix chain)

    p = self.last
    # START at the last state we were at. p is the "current position" for extending.

    while p != -1 and token not in self.states[p].next:
        self.states[p].next[token] = cur
        # STATE p does NOT have a transition on 'token'
        # So ADD the transition: p --token--> cur
        # This means all strings ending at p can extend with token to reach cur
```

```

p = self.states[p].link
# MOVE p backwards via suffix link (shorter context)
# Continue adding transitions at all ancestors that need it

```

Intuition so far: Walk backwards via suffix links, and at each state that lacks a token transition, add a new transition to cur. This ensures all relevant contexts can use the new token.

Python

```

# NOW p == -1 OR token already exists in p.next

if p == -1:
    # We walked all the way back to "before root"
    # This means we added transitions everywhere down the chain
    self.states[cur].link = 0
    # cur's suffix link points to ROOT (state 0)
    # "If you fail matching at cur, try root"

```

Intuition: If we exhausted all suffixes without finding an existing transition, the fallback is root. Any string at cur has a suffix that matches from root.

Python

```

else:
    # p != -1: we stopped because p already has a 'token' transition
    q = self.states[p].next[token]
    # q is the state we'd reach if we followed p --token--> from state p
    # q might be OLD (already existed before this token insertion)

```

Intuition: We found a reusable state q. Now we need to decide: can we link cur to q directly, or do we need to split q?

Python

```

if self.states[p].length + 1 == self.states[q].length:
    # SIMPLE CASE: length[p] + 1 == length[q]
    # This means: "the strings at q are exactly 1 longer than strings at p"
    # This is the EXPECTED structure
    self.states[cur].link = q
    # cur's suffix link points directly to q
    # Both cur and q represent the same equivalence class

```

Intuition: The structure is already perfect. q is the right place for all suffixes of cur. Link them directly.

Python

```
else:
    # COMPLEX CASE: length[p] + 1 != length[q]
    # MISMATCH! q is too long compared to p's length + 1
    # This breaks the automaton invariant
    # SOLUTION: Clone q into a shorter version

    clone = self.expand_state(deepcopy(self.states[q]))
    # CREATE a copy of q (same transitions, same suffix link)
    # clone is a NEW state with all the structure of q

    self.states[clone].length = self.states[p].length + 1
    # FIX clone's length to the CORRECT value
    # Now: length[clone] == length[p] + 1 (exactly what we need)
```

Why clone? Imagine q represents strings like "ABC" and "XYZABC" (both end at q, both have length 6). But p --token--> q where p has length 5. When we add a new token (cur has length 6), we need q to represent only contexts where the predecessor has length 5. But q also handles longer predecessors (length 5 for "ABC" vs other lengths for "XYZABC"). So we split:

- **clone:** handles the "length 5 predecessor" case
- **q:** keeps handling longer/other cases

Python

```
while p != -1 and self.states[p].next[token] == q:
    self.states[p].next[token] = clone
    # REDIRECT p's transition from q to clone
    # p --token--> clone (was p --token--> q)
    # Now p points to the shorter cloned version

    p = self.states[p].link
    # Move p up the suffix chain
    # STOP when: we find a p where next[token] != q
    # (meaning p already transitions differently on token)
```

Intuition: Walk backwards, and redirect all ancestors that were pointing to q to instead point to clone. Stop when we hit one that already points to something else (that ancestor is satisfied differently).

Python

```
self.states[q].link = self.states[cur].link = clone
# NOW reconnect the suffix chain
# q.link = clone (q now falls back to clone)
# cur.link = clone (cur also falls back to clone)
# Both q and cur agree: their common suffix is clone
```

Summary of cloning:

1. We detected q is "too long": its length exceeds expectations
2. We create clone with the correct length
3. We redirect all needy ancestors to clone
4. Both old q and new cur point back to clone as fallback
5. Automaton is balanced again

transfer_state(index, length, token)

This function is for **querying**: given a current state and a token, where do we go next?

Python

```
def transfer_state(self, index: int, length: int, token: int,
                  is_infer=None, step=None, counter=None):
    # index: which state are we currently at?
    # length: how long is the current matched prefix?
    # token: what token do we want to process?
```

Intuition: This is like a DFA (deterministic finite automaton) transition. We're at state index with matched length, and we see token. Where's the next state?

Python

```
if counter != None:
    self.counter = counter
# Update internal counter (used for visualization/debugging)

while index != 0 and token not in self.get_state(index).next:
    # LOOP: current state has NO outgoing transition on 'token'
    # Try the suffix link (shorter context)

    index = self.get_state(index).link
    # JUMP to suffix (fallback state)

    length = self.get_state(index).length
    # UPDATE length to match the suffix state's max length
    # (This is correct because the suffix has a shorter matched prefix)
```

Intuition: If the current state can't handle the token, walk backwards via suffix links until you find one that can. Each jump means "I couldn't match that token, let me try a shorter context."

Python

```
if token in self.get_state(index).next:
    # FOUND IT! Current state (or a suffix) has the transition

    index = self.get_state(index).next[token]
    # FOLLOW the transition
```

```
length += 1
# INCREMENT matched length (we matched one more token)
```

Intuition: We found a state with the token transition. Move to that next state and increment the length.

Python

```
else:
    # No state in the entire suffix chain has this transition
    # Not even root handles it

    index = length = 0
    # RESET to root (index 0, length 0)
    # Matching failed completely
return index, length
```

Intuition: Even root doesn't know this token. Start over from scratch.

gen_draft(index, start_token)

Draft generation

Python

```
def gen_draft(self, index: int, start_token: int):
    endpos = self.get_state(index).min_endpos
    # Get the position where the shortest string at 'index' ends

    pred_ids = [start_token] + self.input_ids[endpos + 1 : endpos + self.n_predicts]
    # EXTRACT prediction: start with the given token, then follow
    # the input sequence from position endpos+1 for n_predicts tokens
```

Intuition: Once you've matched up to a state **index**, `gen_draft` extracts the next `n_predicts` tokens by looking them up in the `input_ids` array starting from where this state's shortest string ends. It's speculative decoding: "Here's what likely comes next based on the pattern we've seen."

Python

```
if len(pred_ids) < self.n_predicts:
    pred_ids.extend([0] * (self.n_predicts - len(pred_ids)))
    # PAD with zeros if we don't have enough predictions

return pred_ids
```

to_anc(index): Find Ancestor with Enough Data

Python

```
def to_anc(self, index: int):
    if index != 0:
        length_to_end = self.max_length - self.get_state(index).min_endpos
        # How many tokens are there from the end of the shortest string
        # at state 'index' to the very end of input?

        while self.get_state(index).link != 0 and self.n_predicts > length_to_end:
            # While we need more tokens (n_predicts > available) and
            # we can still go back via suffix link

            index = self.get_state(index).link
            # Follow suffix link (go to a predecessor state)

            length_to_end = self.max_length - self.get_state(index).min_endpos
            # Recalculate available tokens at this ancestor

    return index
```

Intuition: Walk backwards via suffix links to find a state that has enough tokens after it to generate `n_predicts` predictions. It's insurance: "If my current state doesn't have enough data, go to an ancestor that does."

SAM Construction demo walkthrough

https://docs.google.com/presentation/d/14YMXYuTAMsTs3qc6ga5HvxcQlStUQqRcRAvgdBE6-lc/edit?slide=id.g3e3750a67b4_0_43#slide=id.g3e3750a67b4_0_43

Summary of Static SAM Building Logic

The Static SAM is built incrementally by reading one token at a time and continuously updating the automaton structure.

At a high level, the automaton tries to remember all previously seen token patterns in a compressed graph form. Every new token extends the existing structure by creating new transitions and updating suffix connections.

The construction process mainly does three things:

- **Create a new state for the newly added token**
 - Each new token extends the current sequence length.
 - A new state is added representing substrings ending at that position.
- **Update transitions using suffix links**
 - The algorithm walks backward through smaller suffix contexts.
 - Wherever the current token transition is missing, it adds a transition to the new state.

- This allows multiple previously seen contexts to reuse the same continuation.
- **Clone states only when necessary**
 - Sometimes an existing state becomes too general and cannot correctly represent two different suffix structures.
 - In such cases, the algorithm creates a cloned copy of the state and splits the structure properly.
 - Cloning helps maintain a compact and correct automaton without duplicating the entire graph.

Conceptually, suffix links act like “fallback shortcuts” to smaller matching contexts whenever an exact continuation is not available.

The final automaton efficiently stores repeated token patterns and enables fast lookup and draft generation during speculative decoding.

NEW PROPOSAL

Current Limitation:

In the current implementation of StaticSAM, each SAM state stores only a single representative occurrence position using:

```
Python
min_endpos: int
```

During draft generation, the continuation sequence is generated only from this stored occurrence:

```
Python
pred_ids = input_ids[endpos + 1 : endpos + n_predicts]
```

As a result, every matched SAM state produces only one continuation sequence and therefore only one draft candidate.

Example:

```
input_ids = [A, B, C, A, B, C, D, E]
             0  1  2  3  4  5  6  7
```

Substring “BC” occurs at positions 1 and 4 (both end with endpos=2 and endpos=5)

After "BC" at position 2: next tokens are [A, B, C, D, E] (positions 3-7)

After "BC" at position 5: next tokens are [D, E] (positions 6-7)

If these strings are suffix-equivalent in SAM:

- We could store `top_k_endpos` = [2, 5]
- Generate multiple draft sequences from different continuations

Proposed Extension:

The proposed approach is to store multiple occurrence positions (top-k end positions) for every SAM state instead of only **`min_endpos`**.

Conceptually:

SAM State → Multiple Occurrence Positions → Multiple Continuations → Multiple Draft Sequences

This enables the automaton to propose multiple draft candidates for the same matched substring.

Theoretical Feasibility

This extension is theoretically well aligned with suffix automaton properties.

Each SAM state naturally represents an **endpos equivalence class**, meaning the represented substring may occur at multiple positions in the corpus. Therefore, storing multiple occurrence positions is a natural extension of the existing structure rather than a structural modification to SAM theory.

Required Changes in Static SAM

1. State Structure Modification

Current implementation:

```
Python
min_endpos: int
```

Proposed implementation:

```
Python

topk_endpos: List[int]
```

Each state will maintain multiple representative occurrence positions.

2. State Construction Changes

During token insertion:

```
Python
topk_endpos = [self.max_length]
```

instead of storing a single min_endpos.

3. Clone State Handling

Clone states created during SAM splitting can inherit the same topk_endpos list as the original state.

The existing cloning mechanism remains mostly unchanged.

4. End Position Propagation

After SAM construction, occurrence positions should be propagated through suffix links in decreasing order of state length.

Propagation logic:

Longer States → Shorter Suffix States

For every state:

```
Python
parent.topk_endpos.extend(child.topk_endpos)
```

After merging, only the best k occurrences are retained.

Selection Strategy for Top-k End Positions

The proposed selection criterion is:

Occurrences producing longer continuation sequences

This can be computed using:

$\text{remaining_length} = \text{max_length} - \text{endpos}$

Occurrences with larger remaining suffix lengths are preferred since they enable longer draft generation.

Changes in Draft Generation

Current implementation generates a single draft:

```
Python
gen_draft(index, start_token)
```

Proposed implementation:

```
gen_draft_candidates(index, start_token)
```

which returns:

```
List[List[int]]
```

Each stored end position generates one candidate continuation sequence.

Complexity Analysis

Component	Current	Proposed
State Storage	$O(\text{states})$	$O(k * \text{states})$
Draft Generation	$O(1)$	$O(k)$
Construction Complexity	$O(N)$	Approximately $O(k * N)$
Memory Overhead	Low	Moderate

For small values of k (e.g., 2-8), the additional overhead remains practical.

Potential Advantages

- Enables multi-branch speculative drafting.
- Improves continuation diversity.

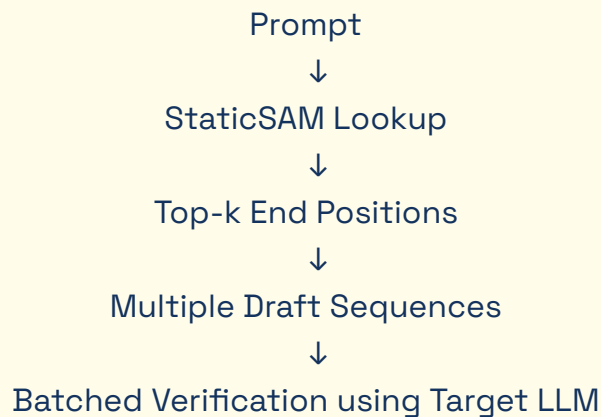
- Utilizes the full occurrence information represented by SAM states.
- Extends Static SAM from single-path retrieval to multi-candidate retrieval.

Parallel Verification of Multiple Draft Sequences

The proposed extension of StaticSAM enables each SAM state to store multiple representative occurrence positions (top-k end positions) instead of a single min_endpos. Consequently, the automaton can generate multiple draft continuations for the same matched substring.

To efficiently verify these multiple draft candidates, all generated drafts can be processed simultaneously using a single batched forward pass through the target language model.

Multi-Draft Verification Pipeline



Batched Verification

Instead of verifying each draft independently, all candidate drafts are packed into a batch:

```
Python
batch_inputs = [
    prompt + draft1,
    prompt + draft2,
    prompt + draft3
]
```

The target model then performs a single batched forward pass.

Shared Prefix Optimization

Most draft candidates generated from the same SAM state are expected to share common prefixes.

Example:

Draft 1: the cat sat

Draft 2: the cat is

Draft 3: the cat ran

This structure can be further optimized using:

- shared KV-cache reuse,
- tree attention,
- branch-based decoding,
- speculative tree verification.

1. What an “endpos” is

- An endpos = index in input_ids where some substring represented by a state ends.
- If a state has multiple endpos values, each endpos gives a different continuation (tokens following that end position).

2. Why keep top-k endpos

- A single min_endpos yields only one draft continuation.
- Multiple endpos allows proposing several candidate continuations that actually appeared in data (improves diversity and recall).
- Keeping only k keeps memory/time bounded.

3. Recording during construction

- When you create a new state for the newly appended token, record a terminal occurrence (state_index, current_max_length). At creation time current_max_length == the position where that state’s shortest string ended, so this record implicitly encodes both an endpos and the length context it represents.
- We append these terminal records as we build the automaton (cheap, $O(1)$ per token).

4. Post-build aggregation (the usual, practical flow)

- Create a small bucket (list) per state and append each recorded endpos into its state’s bucket.
- Process states in decreasing order of length (longest first). For each state:
 - Normalize the bucket (deduplicate + sort) and truncate to k (apply your selection heuristic).
 - Propagate/merge that state’s kept endpos list into its suffix-link parent’s bucket (parents accumulate endpos from children).
- After processing all states, each state’s bucket contains up to k representative endpos drawn from its occurrences and from longer contexts that map to it.

5. Selection heuristic:

- Most recent (largest endpos): favors endpos with the most remaining tokens after them → usually yields longer continuations.

Topk is set to 5 currently

Min_endpos: represented the 1st occurrence of the substring in the corpus

Static sam size according to original implementation: 20gb

Modified SAM: 30gb

Aggregated Results - TopK EndPos Evaluation

Top-K	Steps	Wall Time	Tok/sec	Mean Accepted Tokens
3	40625	752.52s	271.42	5.03
4	40625	619.75s	329.56	5.03
5	40625	615.49s	331.84	5.03
WordGroupAware SAM	40625	750.41s	273.7	5.03

Example:

1 2 3 4 5 6 7 8 9 10 11 12 13 14
The cat sat on the mat. **The cat** stood up. **The cat** ran away.

Min end_pos for “the cat”: 2

Multiple end_pos [2, 8, 12]

Keeping track of multiple endpos and create multiple draft sequences

Paper Structure (Merged))

Word Groups as Linguistic Priors for Indian Language Large Language Models

Summary

This research investigates the hypothesis that word groups act as semantically cohesive linguistic units in Indian languages. We propose improving Large Language Model (LLM) performance through two complementary methods: 'Word-group Augmented Prompting' for explicit structural guidance, and a 'Word-group Aware RoPE' mechanism for implicit representation learning. This framework addresses critical challenges such as rich morphology and suboptimal tokenization, with the goal of enhancing performance in downstream tasks including Named Entity Recognition (NER), slot filling, and machine translation. The proposed methodology will be evaluated against both Indic-specific models (e.g., Param, Sarvam) and strong multilingual baselines (e.g., Gemma, Qwen).

Hypothesis

For Indian languages, word groups can be considered as a semantically cohesive linguistic unit, and incorporating them implicitly (through representations) or explicitly (through prompts) leads to better language understanding and downstream task performances.

1. Introduction

1.1 Motivation

1.2 Hypothesis and Observation

1.3 Contributions:

- a. Word-group Augmented Prompting

- b. Word-group Aware RoPE Mechanism (representation learning)

2. Background / Literature

- a. Indian language-specific challenges: rich morphology, tokenisation etc
- b. Word groups
- c. RoPE
- d. In Context Learning - Few shot prompting, prompt engineering/ linguistically informed prompting
- e. Finetuning with Gemma 4 (31B) (Sangraha(verified)/IndicCorpV2 dataset)

Methodology

3. Word-Group Augmented Prompting

3.1 Motivation

3.2 Prompt Design - variants (WG only, NER only, WG+NER etc.)

4. Word-Group Aware RoPE

4.1 Motivation

4.2 Proposed Method

May consider at least 2 variants

- All tokens in the word group share the same 'theta' (currently working on)
- Some way of hierarchical positioning - word group position + relative token position

4.3 Experiments

- Training LLMs based on 4.2

5. Experimental Setup

a. Models:

Open and Indic-specific: Param, Sarvam, Aya etc

Strong multilingual: Gemma, Qwen

b. Tasks and Datasets

NER, Slot Filling, POS tagging, translation, IndicIE

c. Metrics

6. Results and Analysis, Ablations etc.

RoPE

Extrinsic Evaluation

LSTM+RoPE changes → NER (downstream task)

- With LSTM + RoPE vs regular LLM

- Finetuning NER task
- LSTM followed by LoRA finetuning of LLM (with word groups identified) - perplexity metric (as an intrinsic measure)

New Thread Ideas

1. Simplest: Word-Group Aware MTP

Current MTP predicts: $P(x_{t+1}, x_{t+2}, \dots, x_{t+k} | x_{\leq t})$

independently through parallel heads.

But language is not token-level independent.

Instead predict:

$$P(G_1, G_2, \dots, G_m | x_{\leq t})$$

where G_i are word groups.

Example:

[प्रधान मंत्री] [ने] [नई योजना] [की घोषणा की]

Instead of

प्रधान | मंत्री | ने | नई | योजना | ...

Can have Loss function as $L = L_{LM} + \lambda L_{group}$

$$- L = L_{MTP} + \lambda_1 L_{boundary} + \lambda_2 L_{contrastive}$$

Where L_{group} could be a combination of Boundary Prediction

Loss, next group length prediction, contrastive group loss etc.

Boundary loss- Predict group boundaries.

Contrastive loss- Pull representations of tokens within the same group together and push apart tokens from different groups.

2. Self-Distillation with Word-Group Targets

Current distillation teaches:

$$P_S(y|x) \approx P_T(y|x)$$

Teacher only transfers **what to predict**, not **why those tokens belong together linguistically**.

Example: भारत सरकार ने नई योजना की घोषणा की।

Teacher logits:

भारत

सरकार

ने

नई

योजना

...

Word-group

[भारत सरकार]

[ने]

[नई योजना]

[की घोषणा की]

Idea: Instead of distilling only token probabilities, also distill group structure.

Auxiliary Group Prediction

Teacher - generate normal token distributions

Student - predicts: normal tokens (L_{KD}), word group boundaries (L_{group}). Total: $L = L_{KD} + \lambda L_{group}$

3.

Main Meeting Minutes

Several recent approaches to mitigate

Please note that several of these RL+Reverse distillation approaches let you write your own tools/verifiers which could be our Samanvaya verifiers!

1. ON-POLICY DISTILLATION OF LANGUAGE MODELS: LEARNING FROM SELF-GENERATED MISTAKES:

<https://arxiv.org/pdf/2306.13649>

2. https://zhoutongfu.github.io/zhoutong-ai-blog/posts/on_policy_self_distillation/

3. SELF-DISTILLATION ENABLES CONTINUAL LEARNING : <https://arxiv.org/pdf/2601.19897>

4. Self-Distilled Policy Gradient : <https://arxiv.org/pdf/2606.04036>

5. GLM-5: from Vibe Coding to Agentic Engineering : <https://arxiv.org/pdf/2602.15763>

6. MiMo-V2-Flash Technical Report <https://arxiv.org/pdf/2601.02780>

7. Balancing Forget Quality and Model Utility: A Reverse KL-Divergence Knowledge Distillation Approach for Better Unlearning in LLMs <https://aclanthology.org/2025.naacl-long.60.pdf>
(first work in this thread)

Vocab Diet: Reshaping the Vocabulary of LLMs via Vector Arithmetic

<https://arxiv.org/pdf/2510.17001>

Speculative Decoding Across Languages

<https://arxiv.org/pdf/2605.30580>

AdaSpec: Adaptive Multilingual Speculative Decoding with Self-Synthesized Language-Aware Training and Vocabulary Simplification

<https://ojs.aaai.org/index.php/AAAI/article/view/40307>

RL, Distillation, Sama nvaya

Jun 19, 2026

Working at the tokenization level

- Any way of learning word-groups at the tokenization level.
 - BPE, ULM
 - PickyBPE (<https://arxiv.org/pdf/2409.04599>), BPEKnockout, SuperBPE (<https://arxiv.org/pdf/2503.13423>)
- Teacher-student model
 - Teacher model (trained with word-group/ RoPE-based)
 - Param + RoPE → task: identify/recognise word group?
Prompting param to perform word group recognition?
- RL in Eagle
 - To make draft model to generate word-group aware
- Add word group boundary token to tokenizer <WG>
Example: राम ने <WG> रावण को <WG> मारा <WG>
 - Finetune LM on objective: $L = L_{LM} + \lambda L_{\text{boundary}}$
 - At every inference step, add next token probability and boundary probability
 - Evaluate on group completion rate, boundary prediction accuracy, generation coherence

Jun 22, 2026

Meeting Notes:

- Tokenization Modification with Samanvaya Rules.
- Go through the directions shared by karthika.
- Analyze the tokenization modification according to samanvaya rules.
- Can we have a Student-Teacher OPSD. With the teacher is modified tokenization.
- Can we have a Student-Teacher OPSD. Without modified tokenization. As the rules are or the words are in numbers. so lets say “ram ne ravan ko mara”. With a normal model it will give white space tokenized tokens. Now with **addition of samanvaya rule**, if we modify or find out the tokenized token to some representation, then we might have the student-teacher without modification of tokenizer.

Tokenization

Tokenization

SuperBPE (COLM 2025)

<https://arxiv.org/pdf/2503.13423>

- Introduce a superword tokenization algorithm that produces a vocabulary of both subword and “superword” tokens, which we use to describe tokens bridging more than one word.
- introduces a pretokenization curriculum to the byte-pair encoding (BPE) algorithm
- **whitespace pretokenization** is initially used to enforce learning of subword tokens only (as done in conventional BPE), but it is **disabled in a second stage**, where the tokenizer transitions to learning superword tokens
[SuperBPE tokenizers scale much better with vocabulary size]
- Experiment settings and results
 - pretrain English LMs at 8B scale from scratch (fixed model size, vocabulary size, and training compute)
 - 27% to 33% more efficient at inference time
 - an average improvement of +4.0% over 30 downstream tasks (+8.2% on MMLU, and wins on 25 of the 30 individual tasks)
- Tokenization process used - **original** Sennrich et al.
 - Pretokenization - splitting on whitespaces
 - Text - split into bytes
 - Iterative merges based on most frequent pairs of adjacent tokens (until predecided vocabulary size - reached)
- **SuperBPE Tokenization**
 1. First learns subwords (by using pretokenization to prevent merges across whitespace)
 2. Learns superwords (by lifting this restriction).

Stage 1: equivalent to regular BPE training and continues up to a certain vocabulary size t , which we call the transition point ($t < T$)

Stage 2: tokenizer training resumes from the vocabulary learned thus far, but this time whitespace pretokenization is skipped.

[**Note:** $t = T$ corresponds to BPE, and $t = 0$ corresponds to a naive revision of BPE that foregoes whitespace pretokenization at any point in training.]

Step	BPE	SuperBPE
Initial training (iterations 0 to t)	Whitespace boundaries enforced	Whitespace boundaries enforced
Interactions (t to V)	Whitespace boundaries enforced	Whitespace boundaries removed
Result	Only subwords Eg: by, the, way, in, between	Subwords + superwords Eg: by_the_way, in_between

Transition point - after k number of merges
After transition point merges are considered across whitespaces.

BoundlessBPE (COLM 2025)

By the way
Y_t
By the

<https://arxiv.org/pdf/2504.00178>

At every step, algorithm considers:

- Regular (subword) merge or
- Superword merge

Based on frequency

NOTE: superword merge occurs only when both pretokens are already complete single-token words.

Example

Text: 'To be or not to be'
Baseline: ['To', ' be', ' or', ' not', ' to', ' be']
Boundless BPE: ['To be', ' or not', ' to be']

PickyBPE style deletion
If a token almost always appears as part of a larger merged token, it can be removed from the vocabulary to free space for more useful tokens.

Example:

bet + ween -> between

If ween almost always occurs inside between, delete ween from the vocabulary.

How is the 'complete pretoken' tracked?

Keep the training corpus as:

```
[  
  ['T', 'i', 'p'],  
  [' of'],  
  [' the'],  
  [' h', 'at']  
]
```

where each outer list element is a **pretoken** and the inner list contains the current tokens inside that pretoken.

Possible directions: Tokenization with Samanvaya Word Grouping

1. Direction 1

Boundless BPE:

if pretoken_A is complete and pretoken_B is complete:
 allow supermerge

Instead,

if pretoken_A and pretoken_B belong to same Samanvaya group:
 allow supermerge

~~2. Direction 2~~

~~Instead of treating a complete (space-separated) word as pretoken, start with Samanvaya word group as pretoken (extension of Boundless BPE)~~
~~[Rule-based]~~

3. Direction 3

Instead of hard constraints for merging, modify merge scoring.

Std BPE uses scoring based on frequency: higher $\text{freq}(a,b) \rightarrow$ merge ab

- In addition to frequency, add a term that rewards the word_group based merges as score for merging .

Example: $\text{score}(a,b) = \text{freq}(a,b) \times (1 + \lambda G(a,b))$

$G(a,b)$ - whether a and b belong to the same group

4. Direction 4

RL in tokenizer training [needs more background]

Rewards based on compression and word group coverage

$$R = w_1 * R_{\text{compression}} + w_2 * R_{\text{group}}$$

$$\text{Example: (needs further thought)} R_{\text{group}} = \frac{\# \text{groups represented as single tokens}}{\# \text{groups}}$$

R_{group} could look at group coverage

Direction 1

These are my insights on the first direction

First Direction: if pretoken_A and pretoken_B belong to same Hindi word group: allow supermerge

Step 1: Take Hindi corpus and annotate wordgroups राम ने → [राम#ने]

Step 2: From the annotated corpus extract the wordgroup boundary info

Step 3: Pass the wordgroup boundary info to the tokenizer during training so that the supermerge phase can use it

Step 4: Use the boundary info for supermerge, if the two adjacent tokens are pretokens (whole words) and the second pretoken is a wordgroup boundary token then merge both. So when राम and ने are wordgroup tokens which are adjacent and ने is annotated as a wordgroup end these two pretokens can be merged thus forming a single token

Condition: Only in the case of wordgroup supermerges there is no need of considering frequency as a condition

Observation 1: Even if supermerge happens again after the wordgroup is created the wordgroup is still preserved inside the larger token because no splitting happens after merging

PickyBPE

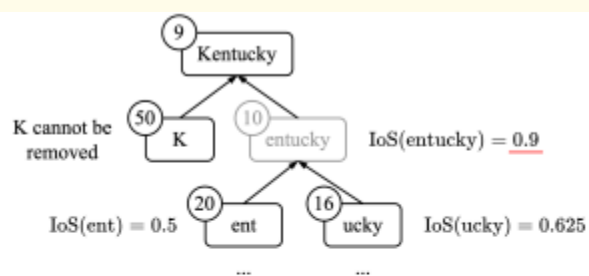


Figure 1: An example of a series of merges to produce the token Kentucky. Pre-merge token frequencies are shown in circles. In the vanilla BPE algorithm, entucky should also be stored in the vocabulary, whereas it is redundant after the merge. In this example, Intersection over Self (IoS) effectively captures the intermediate token, as $\text{IoS}(\text{entucky}) \geq \mathcal{T} = 0.9$.

Distillation

Self-Distillation Enables Continual Learning (2026)

Problem

- Continual learning is difficult because standard supervised fine-tuning (SFT) is off-policy and causes catastrophic forgetting.
- After learning a new task:
new task accuracy $\uparrow$
old capabilities $\downarrow$

Main Idea

Use the model itself as both:

- Teacher
- Student

Teacher receives: Input + Demonstration

Student receives: Input only

Then train the student to imitate the teacher on **student-generated trajectories**.

This converts demonstration learning into **on-policy learning**.

Main Contribution

Introduces **Self-Distillation Fine-Tuning (SDFT)**

Instead of:

Expert Demonstrations



Supervised Fine-Tuning

Use:

Expert Demonstration



Teacher (same model + demonstration)



Student-generated outputs



On-policy distillation

The key novelty is:

Demonstrations are converted into an on-policy learning signal without requiring RL rewards.

Methodology

Teacher policy: $\pi(y | x, c)$

where:

x = input

c = demonstration

Student policy: $\pi_\theta(y|x)$

Student generates: $y \sim \pi_\theta$

Loss: $D_{KL}(\pi_\theta(\cdot|x) \parallel \pi(\cdot|x,c))$ on student-generated trajectories.

On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes ([ICLR 2024](#))

Problem: Traditional KD trains on **dataset sequences** or **teacher-generated sequences**.

- This creates train-inference mismatch.
- During inference, student generates its own outputs, which may be different from training data.

Main Contribution

- Introduces **Generalized Knowledge Distillation (GKD)**
- Train on $y \sim p_s$, instead of $y \sim \text{Dataset}$
- Teacher evaluates student-generated outputs

Methodology

- Student generates: Input $\rightarrow$ Student Output
- Teacher provides token probabilities on those outputs.
- **Loss:** $D(p_T \parallel p_s)$ computed on student trajectories.
- General form: $L_{GKD} = (1-\lambda)L_{\text{supervised}} + \lambda L_{\text{on-policy}}$
where, λ controls how much student-generated data is used

Experimental Setup

- Teacher: T5-XL (~3B)
- Students: T5-small, T5-base, T5-large
- Tasks:
 - Summarization: XSum
 - Translation: WMT
 - Reasoning: GSM8K
 - Instruction following: BBH, MMLU

Results

Compared to Supervised FT, Supervised KD, SeqKD. GKD consistently outperforms them.

Reported gains:

- ~2.1× improvement over baseline KD on summarization
- ~1.7× on translation
- ~1.9× on reasoning

This paper explicitly shows: **GKD + RL** can be combined

$$L = L_{RL} + \alpha L_{GKD}$$

How Word Grouping can fit

Using SDFT, teacher receives Sentence + Word Groups

Example:

[भारत सरकार]

[ने]

[नई योजना]

[की घोषणा की]

Student receives:

भारत सरकार ने नई योजना की घोषणा की

Distill in student:

- boundary locations
- group lengths
- phrase structure

Potential loss: $L = L_{KD} + \lambda L_{\text{boundary}}$

Why EAGLE is not suitable for On-Policy Distillation

- EAGLE is built around a **fixed target LLM**: the draft module is trained on **target features** and a **shifted-token input**, while verification uses the target model's probabilities, and the paper explicitly says the **generated distribution remains unchanged**.
- We cannot expect EAGLE's verifier to enforce grouping automatically, as EAGLE's **acceptance rule is still token-distribution based**.
- Acceptance depends on how often **draft $\neq$ verifier**, not directly on linguistic theory.

Jun 26

Method	Main Idea	Learns Across Words?	Uses Linguistic Knowledge?	Generalization	Need LM Retraining?	Suitability for Samanvaya
Standard BPE	Merge most frequent adjacent tokens within whitespace-delimited pretokens	No	No	Statistical only	Yes (if tokenizer changed)	Baseline

SuperBPE	Learn subwords first, then allow merges across whitespace in a second stage	Yes	No	Learns frequent multi-word expressions	Yes	Good starting point
Boundless BPE	During training, allow supermerges between adjacent complete pretokens based on frequency	Yes	No	Better utilization of vocabulary through common superwords	Yes	Best framework to inject Samanvaya rules
VocabDiet	Restructure vocabulary compositionally using base words + transformation vectors instead of changing tokenization	N/A	Linguistic morphology	Better vocabulary utilization	No (lightweight adaptation possible)	Inspiration only; not directly applicable to word groups

Proposal: Samanvaya-Guided Tokenization for Hindi Language Models

Motivation

Existing BPE-based tokenizers (BPE, SuperBPE, BoundlessBPE) are entirely driven by corpus statistics and token frequencies. They are unaware of linguistic units such as Samanvaya word groups. Consequently, meaningful Hindi word groups may be split across multiple tokens even when they function as a single semantic unit.

Our objective is to investigate whether incorporating Samanvaya word-group information during tokenizer training can produce a more linguistically meaningful vocabulary while maintaining the statistical strengths of BPE.

Direction 1: Samanvaya-Guided Supermerge (Hard Constraint)

Research Hypothesis

If two adjacent pretokens belong to the same Samanvaya word group, they should be allowed to form a superword token irrespective of their corpus frequency.

Methodology

Step 1

Take a large Hindi corpus.

Apply the Samanvaya word-grouping rules to obtain annotated sentences.

Example

राम ने रावण को मारा -> [राम ने] [रावण को] [मारा]

Step 2

Extract word-group boundary information.

Example

राम → Group 1

ने → Group 1 (boundary)

रावण → Group 2

को → Group 2 (boundary)

मारा → Group 3 (boundary)

The annotation stores which adjacent pretokens belong to the same linguistic group.

Step 3

Train a BoundlessBPE tokenizer as usual.

Regular BPE merges continue exactly as in the original algorithm.

No modification is made to Stage-1 BPE.

Step 4

During the supermerge phase, use the extracted boundary information.

If

- pretoken A is complete,
- pretoken B is complete,
- both belong to the same Samanvaya word group,

allow a supermerge.

Example

राम + ने -> राम_ने

Similarly,

रावण + को -> रावण_को

Frequency is ignored only for these linguistically valid supermerges.

Observation

Once a Samanvaya word group has been merged into a token, later supermerges preserve the complete group.

Example

राम_ने + रावण_को -> राम_ने_रावण_को

The original word-group boundaries are never broken because tokenizer training performs only merge operations.

Potential Concern

Ignoring frequency may allocate vocabulary entries to very rare word groups.

This may reduce vocabulary efficiency by assigning limited vocabulary slots to infrequent linguistic units.

Direction 3: Samanvaya-Guided Merge Scoring (Soft Constraint)

Motivation

Instead of forcing every Samanvaya merge, incorporate linguistic knowledge as a soft prior during tokenizer learning.

Research Hypothesis

The tokenizer should balance

- statistical evidence from the corpus
- linguistic evidence from Samanvaya

rather than relying solely on either one.

Methodology

Step 1

Generate Samanvaya annotations for the training corpus exactly as in Direction 1.

Step 2

During tokenizer training, every candidate merge receives a modified score.

Instead of

Merge Score = Frequency

use

$$\text{Merge Score} = \text{Frequency} + \lambda \times \text{WordGroupBonus}$$

where

- Frequency is the standard BPE merge count.
 - WordGroupBonus is assigned if the candidate merge belongs to the same Samanvaya word group.
 - λ controls the influence of linguistic knowledge.
-

Step 3

Tokenizer training proceeds exactly like BoundlessBPE.

At every iteration,

- compute regular merge scores,
- compute supermerge scores,
- select the candidate with the highest modified score.

No additional constraints are imposed.

Interpretation

$\lambda = 0$ -> Standard BoundlessBPE

Small λ -> Corpus statistics dominate while linguistic information provides guidance.

Large λ -> Behavior approaches Direction 1, where Samanvaya-guided merges become increasingly preferred.

Advantages

- Preserves the statistical nature of BPE.
 - Avoids allocating vocabulary to every rare word group.
 - Allows systematic ablation by varying λ .
 - Provides a unified framework encompassing both purely statistical and linguistically guided tokenization.
-

Evaluation Plan

Intrinsic Tokenizer Evaluation

- Vocabulary utilization
 - Average sequence length
 - Bytes per token / compression ratio
 - Percentage of complete Samanvaya word groups represented by single tokens
 - Coverage of unseen word groups
 - Vocabulary distribution
-

Language Model Evaluation

Train or continually pretrain a language model using the modified tokenizer and compare with:

- Standard BPE
- SuperBPE
- BoundlessBPE

using

- Perplexity
 - Downstream Hindi benchmarks
 - Inference efficiency
 - Generation quality
-

Expected Contribution

- Introduce linguistic priors into tokenizer learning using Samanvaya word-group rules.
- Study the trade-off between statistical tokenization and linguistically informed tokenization.
- Propose both a hard-constraint (Direction 1) and a soft-scoring (Direction 3) framework for integrating word-group information into BoundlessBPE.
- Analyze whether linguistically guided tokenization improves tokenizer quality and downstream language modeling for Hindi.

